

Day 15: Databricks Fundamentals

5-line beginner summary

1. **Databricks is a cloud platform for data engineering, analytics, machine learning, and GenAI.**
 2. It helps enterprises store, clean, process, analyze, and use large-scale data in one place.
 3. Its main architecture is called a **Lakehouse**, which combines the flexibility of a data lake with the reliability of a data warehouse.
 4. Databricks uses important components like **Workspace, Notebooks, Clusters, Jobs, Delta Lake, Unity Catalog, Databricks SQL, and MLflow.**
 5. For AI/GenAI, Databricks supports model training, model serving, RAG, agents, GenAI evaluation, and governed AI applications. Databricks docs describe it as an integrated platform for ML lifecycle work, from data preparation to production monitoring.
([Databricks Documentation](#))
-

1. What Databricks is

Databricks is a unified data and AI platform.

In simple words, it is a cloud-based environment where data engineers, data analysts, data scientists, ML engineers, and GenAI engineers can work together.

You can use Databricks to:

- Ingest raw data.
- Clean and transform data.
- Store data in Delta Lake tables.
- Run SQL analytics.
- Build dashboards.
- Train ML models.
- Track experiments using MLflow.
- Build GenAI apps and agents.
- Deploy models and AI applications.

Databricks has core components like **workspace, notebooks, jobs, compute/clusters, SQL warehouses, experiments, models, and Delta tables**. The official Databricks component guide defines a workspace as an environment for accessing Databricks assets, and notebooks as web-based interfaces for creating data science and ML workflows.
([Databricks Documentation](#))

2. Why enterprises use Databricks

Enterprises use Databricks because they usually have data coming from many systems:

- Banking systems
- Customer apps
- Logs

- APIs
- CRM
- ERP
- IoT devices
- Documents
- Data warehouses
- Data lakes

Without a platform like Databricks, teams often work separately:

Data engineers use one tool
 Data analysts use another tool
 Data scientists use another tool
 ML engineers use another tool
 GenAI teams use another tool

Databricks tries to bring these workloads into one platform.

Enterprise benefits

Need	How Databricks helps
Large-scale data processing	Uses Spark-based distributed compute
Reliable data storage	Uses Delta Lake
SQL analytics	Provides Databricks SQL and SQL warehouses
ML lifecycle	Integrates with MLflow
Governance	Uses Unity Catalog
GenAI apps	Supports GenAI apps, agents, model serving, evaluation
Automation	Uses Jobs and Pipelines
Collaboration	Notebooks, Git folders, dashboards, experiments

Unity Catalog is especially important in enterprise environments because it provides governance for data and AI assets, including access control, lineage, auditing, discovery, and AI governance. ([Databricks Documentation](#))

3. Lakehouse concept

A **Lakehouse** combines two older ideas:

Data Lake + Data Warehouse = Data Lakehouse

Data Lake

A data lake stores large amounts of raw data cheaply.

Example:

CSV files

JSON files

Parquet files

Images

Logs

Documents

Streaming data

Problem: Data lakes can become messy if governance, quality, and structure are weak.

Data Warehouse

A data warehouse stores clean, structured data for reporting and analytics.

Example:

Sales reports

Finance dashboards

Customer analytics

Monthly business KPIs

Problem: It can be expensive and less flexible for unstructured or ML-heavy workloads.

Lakehouse

A lakehouse gives you both:

Cheap scalable storage + reliable analytics + ML/AI support

Databricks defines a data lakehouse as a system that combines benefits of data lakes and data warehouses, helping organizations avoid isolated systems for BI and ML, establish a single source of truth, and keep data fresh. ([Databricks Documentation](#))

4. Workspace

A **Databricks Workspace** is like your project environment.

It contains:

- Notebooks
- Folders
- Dashboards
- Experiments
- Jobs
- Queries
- Git folders
- Access to compute
- Access to data objects

Easy example

Imagine your IBM project team is building an internal policy Q&A system.

Your workspace may contain:

```
/Policy-RAG-Project
  /01_ingestion_notebook
  /02_chunking_embedding_notebook
  /03_vector_search_notebook
  /04_model_evaluation_notebook
  /dashboards
  /jobs
  /experiments
```

The workspace is where teams collaborate.

5. Notebooks

A **notebook** is an interactive document where you can write and run code.

Databricks notebooks can contain:

- Python code
- SQL code
- Scala code
- R code
- Markdown notes
- Charts
- Visualizations

Simple notebook example

```
df = spark.read.csv("/mnt/sales/sales.csv", header=True, inferSchema=True)
display(df)
```

Then in another cell:

```
SELECT region, SUM(amount)
FROM sales_table
GROUP BY region
```

Why notebooks are useful

They are good for:

- Data exploration
- EDA
- ETL development
- ML experiments
- RAG experiments
- Debugging data issues
- Sharing analysis with team members

Databricks describes notebooks as web-based interfaces that support runnable commands, visualizations, and narrative text for data science and ML workflows. ([Databricks Documentation](#))

6. Clusters

A **cluster** is the compute power used to run your code.

Simple meaning:

Notebook = where you write code Cluster = machine power that runs the code

In Databricks, compute can run Spark jobs, Python code, SQL workloads, ML training, or data pipelines.

Types of compute

Compute type	Used for
All-purpose cluster	Interactive notebook development
Job cluster	Automated production jobs
SQL warehouse	Databricks SQL queries and dashboards
Serverless compute	Managed compute without manually managing cluster details

Databricks documentation distinguishes interactive workloads on all-purpose clusters from automated workloads on job clusters. ([Databricks Documentation](#))

Easy example

During development:

Use all-purpose cluster

In production:

Use job cluster

Because job clusters start for the job, run the task, and shut down. This helps control cost.

7. Jobs

A **Databricks Job** is used to schedule and automate work.

Example:

Run sales ETL pipeline every day at 2 AM Train churn model every Sunday Refresh dashboard every morning Generate embeddings when new documents arrive
--

Databricks Lakeflow Jobs provide workflow automation for Databricks workloads and can coordinate multiple tasks such as ETL workflows, notebooks, ML workflows, and integrations with external systems. ([Databricks Documentation](#))

Example job workflow

Task 1: Ingest raw sales data

Task 2: Clean and validate data

Task 3: Create aggregated table

Task 4: Refresh dashboard

Task 5: Trigger ML model scoring

8. Data ingestion

Data ingestion means bringing data into Databricks.

Sources can be:

- CSV files
- JSON files
- Parquet files
- APIs
- Databases
- Kafka streams
- Cloud storage like S3, ADLS, or GCS
- Business applications
- Logs
- Documents

Easy example

A retail company receives daily sales files.

Source: sales_2026_07_08.csv

Destination: Databricks raw table

Data ingestion flow:

Raw file → Databricks → Delta table → Clean table → Business report

Databricks supports features for loading data into the lakehouse, including Auto Loader for incremental cloud file ingestion, COPY INTO for SQL-based incremental loading, and Lakeflow Spark Declarative Pipelines for ETL pipelines. ([Databricks Documentation](#))

9. ETL and ELT

Both ETL and ELT are data processing patterns.

ETL: Extract, Transform, Load

Extract data → Transform outside target system → Load clean data

Example:

Take raw sales data
Clean it in a processing engine
Load clean data into warehouse

ELT: Extract, Load, Transform

Extract data → Load raw data first → Transform inside Databricks

Example:

Load raw sales files into Delta Lake
Then clean and transform using Spark/SQL

In Databricks, ELT is very common

Because Databricks can store raw data and also process it at scale.

A common pattern is **medallion architecture**:

Bronze → Raw data
Silver → Cleaned and validated data
Gold → Business-ready data

Databricks lakehouse documentation describes this staged improvement pattern as medallion architecture, where data is incrementally refined as it moves through layers. ([Databricks Documentation](#))

10. Databricks SQL

Databricks SQL allows analysts and engineers to query lakehouse data using SQL.

It is useful for:

- BI dashboards
- Business reporting
- Ad-hoc analysis
- Data validation
- KPI calculations
- Analyst self-service

Example

```
SELECT
    region,
    SUM(sales_amount) AS total_sales,
    COUNT(*) AS order_count
FROM gold_sales
```

```
GROUP BY region  
ORDER BY total_sales DESC;
```

Easy business example

A manager asks:

| Which region had the highest sales this month?

A data analyst can use Databricks SQL to query the Gold table and create a dashboard.

Databricks reference architecture describes Databricks SQL as the data warehouse capability powered by SQL warehouses, used for BI and analytics workloads. ([Databricks Documentation](#))

11. Databricks for ML

Databricks is widely used for machine learning because it supports the full ML lifecycle:

```
Data preparation  
Feature engineering  
Model training  
Experiment tracking  
Model registry  
Model deployment  
Monitoring
```

ML example

Suppose a telecom company wants to predict customer churn.

Databricks can be used for:

1. Load customer data
2. Clean missing values
3. Create features
4. Train churn model
5. Track model using MLflow
6. Register model
7. Serve model as REST endpoint
8. Monitor performance

Databricks ML documentation says the platform supports building, deploying, and managing ML applications, unifying the ML lifecycle from data preparation to production monitoring. It also includes Databricks Runtime for ML with common ML libraries and MLflow tracking for experiments and model lifecycle management. ([Databricks Documentation](#))

12. Databricks for GenAI

Databricks can also be used for GenAI applications.

Common GenAI use cases:

- RAG over enterprise documents

- Internal policy assistant
- Customer support chatbot
- SQL question-answering assistant
- Document summarization
- Contract analysis
- AI agents that use tools
- GenAI evaluation and monitoring

Example: RAG system in Databricks

Documents → Chunking → Embeddings → Vector Search → LLM → Answer with citations

Databricks GenAI capabilities include tools for building GenAI apps, agents, tools, and models, and support integration with open-source GenAI frameworks, custom tools, and MCP servers. ([Databricks Documentation](#))

13. Databricks Apps

Databricks Apps allow teams to build and deploy custom applications inside the Databricks environment.

Simple meaning:

You can build an app on top of Databricks data and APIs.

Example apps:

- Sales dashboard app
- Data quality monitoring app
- Internal GenAI chatbot
- ML prediction app
- Policy Q&A app
- Agent-based assistant

Databricks documentation describes Databricks Apps as providing flexible app development and deployment to authenticated users, and custom apps can be powered by Databricks APIs. ([Databricks Documentation](#))

Easy example

A company builds a GenAI app:

User asks question:

"What is our leave policy?"

App calls:

Databricks Vector Search + LLM endpoint + policy Delta tables

App returns:

Answer with source document reference

14. Databricks Agents

Databricks Agents are used to build AI systems that can reason, call tools, retrieve information, and complete multi-step tasks.

Simple difference:

Normal chatbot:

User asks → LLM answers

Agent:

User asks → Agent plans → Uses tools/data/models → Checks result → Answers

Example agent

User asks:

| Find customers with high churn risk and draft a retention strategy.

Agent may:

1. Query customer table
2. Use churn model endpoint
3. Analyze high-risk customers
4. Generate explanation
5. Suggest next-best actions

Databricks provides tools for building, deploying, and using AI agents, including AI Playground for prototyping and MLflow for GenAI tracing, evaluation, and human feedback. ([Databricks Documentation](#))

15. How Databricks connects with MLflow and Delta Lake

This is very important for interviews.

Delta Lake connection

Delta Lake is the storage foundation.

It stores reliable tables in the lakehouse.

Delta Lake provides:

- ACID transactions
- Scalable metadata
- Batch and streaming support
- Schema evolution
- Time travel
- Merge/upsert support
- Reliable data pipelines

Databricks describes Delta Lake as the optimized storage layer for lakehouse tables. It extends Parquet with a file-based transaction log for ACID transactions and scalable

metadata handling, and Delta is the default table format on Databricks. ([Databricks Documentation](#))

MLflow connection

MLflow manages the ML and GenAI lifecycle.

MLflow helps with:

- Tracking experiments
- Logging parameters
- Logging metrics
- Saving models
- Comparing runs
- Registering models
- Deploying models
- Evaluating GenAI apps
- Tracing agent behavior

MLflow 3 for GenAI supports tracking, evaluation, observability, trace logging, scorers, human feedback, and version tracking for GenAI apps and agents. ([Databricks Documentation](#))

Simple relationship

Delta Lake = stores trustworthy data
MLflow = tracks models and experiments
Databricks = platform connecting data, ML, GenAI, jobs, governance
Unity Catalog = governance layer for data and AI assets

Easy business examples

Example 1: Sales analytics

Problem:
Business wants daily sales dashboard.

Databricks solution:
Ingest sales files → Clean data → Store in Delta Lake → Create Gold table → Query using Databricks SQL → Build dashboard

Example 2: Customer churn ML

Problem:
Telecom company wants to predict which customers may leave.

Databricks solution:
Load customer data → Feature engineering → Train ML model → Track with MLflow → Register model → Serve prediction endpoint

Example 3: HR policy GenAI assistant

Problem:

Employees ask many HR policy questions.

Databricks solution:

Load HR documents → Chunk documents → Create embeddings → Store/search relevant chunks → Use LLM → Return grounded answer

Example 4: Agent for business analysis

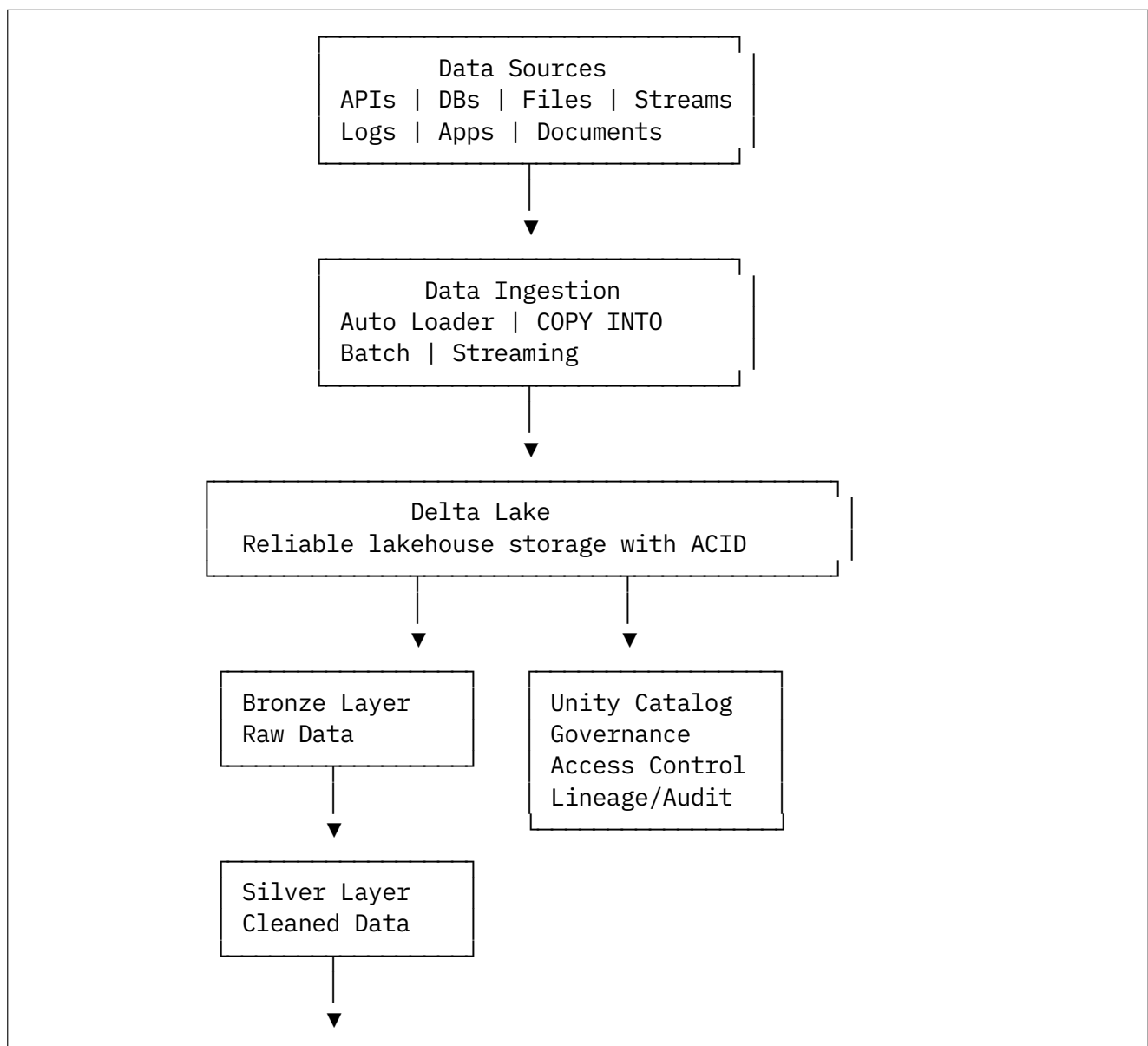
Problem:

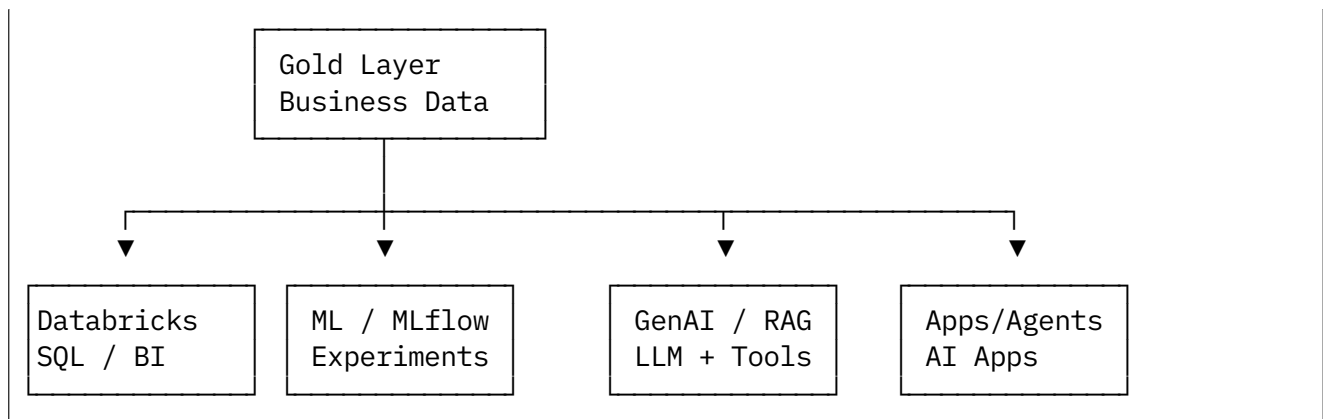
Manager asks: "Why did sales drop in North region?"

Databricks agent:

Queries sales table → Checks product data → Checks customer feedback → Summarizes possible reasons

ASCII diagram: Databricks Lakehouse Architecture





Pseudocode: Databricks data pipeline

START

1. Create Databricks workspace
2. Configure Unity Catalog permissions
3. Create cluster or use serverless compute
4. Ingest raw data
 READ files from cloud storage
 WRITE data into Bronze Delta table
5. Clean data
 READ Bronze table
 REMOVE duplicates
 HANDLE missing values
 VALIDATE schema
 WRITE cleaned data into Silver Delta table
6. Create business-ready data
 READ Silver table
 JOIN with reference tables
 AGGREGATE metrics
 WRITE final data into Gold Delta table
7. Run analytics
 USE Databricks SQL
 CREATE dashboard from Gold table
8. Train ML model
 READ Gold table
 SPLIT train/test data
 TRAIN model
 LOG parameters, metrics, and model using MLflow
9. Register model
 SAVE best model in model registry
10. Serve model or GenAI app
 DEPLOY model endpoint or app

MONITOR quality, drift, and usage

11. Schedule production workflow
CREATE Databricks Job
RUN pipeline daily or hourly

END

Pseudocode-style PySpark example

```
# Step 1: Read raw sales data
raw_df = spark.read.format("csv") \
    .option("header", "true") \
    .option("inferSchema", "true") \
    .load("/mnt/raw/sales/")

# Step 2: Write Bronze table
raw_df.write.format("delta") \
    .mode("append") \
    .saveAsTable("retail.bronze.sales_raw")

# Step 3: Clean data for Silver table
silver_df = raw_df.dropDuplicates() \
    .filter("sales_amount IS NOT NULL") \
    .filter("customer_id IS NOT NULL")

silver_df.write.format("delta") \
    .mode("overwrite") \
    .saveAsTable("retail.silver.sales_clean")

# Step 4: Create Gold aggregation
gold_df = silver_df.groupBy("region") \
    .sum("sales_amount") \
    .withColumnRenamed("sum(sales_amount)", "total_sales")

gold_df.write.format("delta") \
    .mode("overwrite") \
    .saveAsTable("retail.gold.region_sales")
```

How to remember Databricks components

Component	Simple meaning	Interview-friendly explanation
Workspace	Project environment	Place where users access notebooks, jobs, dashboards, experiments, and data
Notebook	Interactive coding document	Used for development, EDA, ETL, ML, and GenAI experiments

Component	Simple meaning	Interview-friendly explanation
Cluster	Compute engine	Runs notebooks, Spark jobs, ML training, and transformations
Job	Scheduler/orchestrator	Automates workflows like ETL, ML training, or dashboard refresh
Delta Lake	Reliable storage layer	Adds ACID reliability and scalable metadata to lakehouse tables
Unity Catalog	Governance layer	Manages access, lineage, audit, discovery, and AI governance
Databricks SQL	SQL analytics layer	Used for BI, dashboards, and business queries
MLflow	ML lifecycle tool	Tracks experiments, metrics, models, registry, deployment, and GenAI evaluation
Databricks Apps	App deployment	Build authenticated apps on top of Databricks data and APIs
Databricks Agents	Agentic AI layer	Build AI agents that use data, tools, models, and evaluation

Common mistakes

1. Thinking Databricks is only Spark

Databricks started strongly around Spark, but today it is much more than Spark.

It supports:

Data engineering
SQL analytics
ML
MLOps
GenAI
Agents
Apps
Governance

2. Confusing data lake and lakehouse

A data lake is mostly storage.

A lakehouse adds:

Reliability
Governance
SQL performance
ML support

Data quality Transaction support

3. Using notebooks directly as production pipelines

Notebooks are good for development, but production should use:

Jobs Pipelines Git Testing Monitoring Access control

4. Not managing cluster cost

Leaving all-purpose clusters running can increase cost.

Better production practice:

Use job clusters or serverless where appropriate Set auto-termination Monitor usage

5. Ignoring Unity Catalog

For enterprise AI projects, governance is not optional.

You need to control:

Who can access data Who can access models Which tables contain sensitive data Where data came from Who queried what

6. Not using Delta Lake properly

Saving everything as plain CSV or Parquet may miss Delta benefits.

Use Delta tables for:

ACID transactions Upserts Schema evolution Time travel Streaming and batch reliability
--

7. Not tracking ML experiments

Without MLflow, teams may lose track of:

Which model version was trained Which parameters were used Which dataset was used

Which metric was best
Which model went to production

8. Treating GenAI apps like normal chatbots

Enterprise GenAI needs:

Grounding
Retrieval quality
Evaluation
Tracing
Human feedback
Access control
Monitoring
Cost tracking

MLflow for GenAI is relevant here because it supports tracing, evaluation, scorers, human feedback, and production observability for GenAI apps and agents. ([Databricks Documentation](#))

Interview-ready summary

You can explain Databricks like this:

Databricks is a unified data and AI platform used to build lakehouse-based data, analytics, ML, and GenAI solutions. Data is usually ingested into Delta Lake tables using a Bronze, Silver, and Gold architecture. Engineers use notebooks and jobs for development and automation, analysts use Databricks SQL for BI, and data scientists use MLflow for experiment tracking and model lifecycle management. Unity Catalog provides governance for data and AI assets. For GenAI, Databricks supports RAG, model serving, GenAI evaluation, Apps, and AI agents.